

Serveur

Le serveur est une API qui reçoit les données du client 1. Le client va ensuite vouloir récupérer par API les messages du client 2.

Recevoir des données

Pour recevoir des données en API, on utilise généralement **POST**. Les données envoyées et reçues dans une API sont toujours en JSON. On utilise donc `jsonify` de `Flask`.

- To send data to the REST API server, you need to make an HTTP POST request and specify the request body in JSON format. In this example, we send a POST request to a ReqBin API endpoint.
- Créer une API rapidement avec Flask
- Post JSON Example

Nous définissons donc une méthode dans notre fonction :

```
@app.route("/receive_data", methods=["POST"])
def receive_data():
    data = request.get_json()
    return jsonify(data)
```

Nous avons comme retour :

```
C:\Users\Pichon>curl -X POST -H "Content-Type: application/json" -d '{"key":"value"}' http://localhost:5000/receive_data
{"key":"value"}
```

Il faut donc établir les besoins à recevoir afin de traiter les données et les stocker :

- ID
- name
- Date d'envoi (Année, Mois, Jours, Heure, Minute, Seconde)
- Date de réception
- Message
- Destination

```
C:\Users\Pichon>curl -X POST -H "Content-Type: application/json" -d '{"ID": 78914, "date": "09:11:03 07/19/25", "message": "Salut ça va ?"}' http://localhost:5000/receive_data
{"status": "receive"}
```

```
C:\Users\Pichon>curl -X POST -H "Content-Type: application/json" -d '{"ID": 78914, "date": "09:11:03 07/19/25", "mesge": "Salut ça va ?"}' http://localhost:5000/receive_data
{"status": "Data in not valid"}
```

```
try:
    # Extract the data sent
    id_user = data["ID"]
    date_user = data["date"]
    date_receive = datetime.datetime.now().strftime("%X %x")
```

```

message = data["message"]
destination = data["to"]

return jsonify({"status": "receive"})

except:
    return jsonify({"status": "Data in not valid"})

```

```

curl -X POST -H "Content-Type: application/json" -d '{"ID": 78914,
"date": "09:11:03 07/19/25", "message": "Salut ça va ?", "to":
"995"}' http://localhost:5000/receive_data

```

Il faut également vérifier que l'ID != destination

```

C:\Users\Pichon>curl -X POST -H "Content-Type: application/json" -d '{"ID": 789, "date": "09:11:03 07/19/25", "message": "Oui et toi ?", "to": "995"}' http://localhost:5000/receive_data
{"status": "ERROR - Your ID and destination it's same"}

```

Stockage des données

Le stockage de données de ce type est souvent en JSON (très utilisé dans le back-end et natif dans les API). Nous avons donc besoin de créer un fichier par conversation entre deux clients. Nous créons donc `"./stockage/{ID}_{to}_CONV.json"`.

Mais il faut un fichier unique par conversation (et pas un dans chaque sens). Nous allons donc mettre l'ID le plus élevé en premier.

```

if ID > to:
    filename = fr"./stockage/{ID}_{to}_CONV.json"
else:
    filename = fr"./stockage/{to}_{ID}_CONV.json"

```

```

curl -X POST -H "Content-Type: application/json" -d '{"ID": 789, "date":
"09:11:03 07/19/25", "message": "Salut ça va ?", "to": "995"}'
http://localhost:5000/receive_data

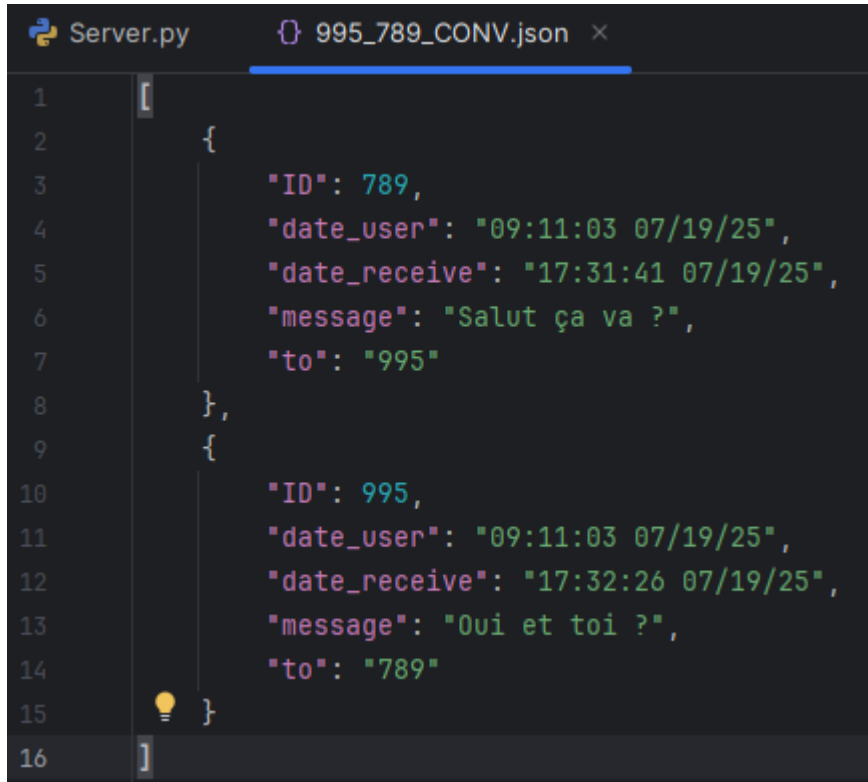
```

```

curl -X POST -H "Content-Type: application/json" -d '{"ID": 995, "date":
"09:11:03 07/19/25", "message": "Oui et toi ?", "to": "789"}'
http://localhost:5000/receive_data

```

Nous avons donc un *échange* entre deux ID mais un même fichier:



```

1  [
2      {
3          "ID": 789,
4          "date_user": "09:11:03 07/19/25",
5          "date_receive": "17:31:41 07/19/25",
6          "message": "Salut ça va ?",
7          "to": "995"
8      },
9      {
10         "ID": 995,
11         "date_user": "09:11:03 07/19/25",
12         "date_receive": "17:32:26 07/19/25",
13         "message": "Oui et toi ?",
14         "to": "789"
15     }
16 ]

```

Récupérer des données

Nous utiliserons la méthode *GET*, avec en argument dans l'URL les deux ID clients (les deux sens fonctionnent). De la forme : `/getdata/955_789`. Mais ce n'est pas la bonne forme d'API. Nous reprenons donc la même forme que le *POST*.

- Il faut vérifier que la forme est bonne (regex)
- Il faut vérifier que le fichier existe

Et par la suite, authentifier via une clé que la personne est autorisée à lire la conversation.

```

if re.fullmatch(r'^\d+_\d+$', conv):
    c1 = str(conv).split("_")[0]
    c2 = str(conv).split("_")[1]
    filename = fr".\stockage\{c1}_{c2}_CONV.json"
    if os.path.exists(filename):
        with open(filename, 'r') as r:
            all_conv = json.load(r)
            return jsonify(all_conv)
    else:
        return jsonify({"status": "This file not exist"})
else:
    return jsonify({"status": "error"})

```



Le point faible majeur est donc la sécurité. N'importe qui possédant 2 ID est en capacité de récupérer le contenu.

```
curl -X GET -H "Content-Type: application/json" -d "{\"conv\": \"995_789\"}"
http://localhost:5000/get_data/
```

Ce qui retourne :

```
[
  {
    "ID": 789,
    "date_receive": "18:27:03 07/19/25",
    "date_user": "09:11:03 07/19/25",
    "message": "Salut \u00c3\u00a7a va ?",
    "to": "995"
  },
  {
    "ID": 995,
    "date_receive": "18:27:15 07/19/25",
    "date_user": "09:11:03 07/19/25",
    "message": "Oui et toi ?",
    "to": "789"
  }
]
```

On génère une clé via OpenSSL `openssl rand -base64 32`. Pour améliorer la sécurité, on ajoute donc une clé qui est rattachée à des discussions.

```
{
  "3uLRF0GpifPqRK19nmS0p3QQ/upYvdSERwRrsb4C3N8=": "995_789"
}
```

On fait ensuite une comparaison pour voir si la clé a accès (et si elle existe) :

```
conv = data["conv"]
key = data["key"]

with open(r'.\stockage\Access.json', 'r') as f:
    access = json.load(f)
    access = access[0]

if key in access:
    if re.fullmatch(r'^\d+_\d+$', conv):
        c1 = str(conv).split("_")[0]
        c2 = str(conv).split("_")[1]
        if access[key] == f"{c1}_{c2}":
```

```
C:\Users\Pichon>curl -X GET -H "Content-Type: application/json" -d '{"conv": "\789_995", "key": "\3uLRF0GpifPqRK19nmS0p3QQ/upYvdSERwRrsb4C3N8="}' http://localhost:5000/get_data/
{"status": "You don't have access for this conversation, attempt is report"}

C:\Users\Pichon>curl -X GET -H "Content-Type: application/json" -d '{"conv": "\995_789", "key": "\3uLRF0GpifPqRK19nmS0p3QQ/upYvdSERwRrsb4C3N8="}' http://localhost:5000/get_data/
{"status": "This file not exist"}

C:\Users\Pichon>curl -X GET -H "Content-Type: application/json" -d '{"conv": "\955_789", "key": "\3uLRF0GpifPqRK19nmS0p3QQ/upYvdSERwRrsb4C3N8="}' http://localhost:5000/get_data/
{"ID": 789, "date_receive": "18:27:03 07/19/25", "date_user": "09:11:03 07/19/25", "message": "Salut \u00c3\u00a7a va ?", "to": "995"}, {"ID": 995, "date_receive": "18:27:15 07/19/25", "date_user": "09:11:03 07/19/25", "message": "Oui et toi ?", "to": "789"}, {"ID": 789, "date_receive": "20:04:54 07/19/25", "date_user": "09:11:03 07/19/25", "message": "Salut \u00c3\u00a7a va ?", "to": "995"}, {"ID": 995, "date_receive": "20:04:55 07/19/25", "date_user": "09:11:03 07/19/25", "message": "Oui et toi ?", "to": "789"}
```

Logs

Les logs permettent de faire un suivi des messages plus simplement que du JSON. On fait donc :

```
def logs_CSV(data: list):
    ID, to = data[0], data[4]
    if int(ID) > int(to):
        filename = fr".\stockage\{ID}_{to}_LOGS.csv"
    elif int(ID) < int(to):
        filename = fr".\stockage\{to}_{ID}_LOGS.csv"
    elif int(ID)==int(to):
        filename = fr".\stockage\ERROR-SAME_LOGS.csv"

    with open(filename, "a", newline='') as f:
        (csv.writer(f)).writerow(data)
```

Pour l'instant, la table n'est pas très intéressante, mais la fonction existe pour la suite. Ensuite, il faut également logger les tentatives de GET :

```
def logs_sending_CSV(key, message, date):
    with open(fr".\stockage\send_error.csv", "a", newline='') as f:
        data = [key, message, date]
        (csv.writer(f)).writerow(data)
```

C'est la fonction qui envoie les données (en fonction de l'erreur et du cas).

	C1 ▾	÷	C2 ▾	÷	C3 ▾	÷
1	3uLRF0GpifPqRK19nmS0p3QQ/upYvdSERwRrsb4C3N8=		Request sucessfull for : 995_789		21:47:07 07/19/25	
2	3uLRF0GpifPqRK19nmS0p3QQ/upYvdSERwRrsb4C3N8=		Tentative on 95_789, not access		21:47:20 07/19/25	
3	3uLRF0GpifPK19nmS0p3QQ/upYvdSERwRrsb4C3N8=		This key note exist		21:47:37 07/19/25	
4	3uLRF0GpifPqRK19nmS0p3QQ/upYvdSERwRrsb4C3N8=		Tentative on 99_789, not access		21:49:14 07/19/25	
5	abc		This key note exist		21:49:56 07/19/25	
6	abc		This key note exist		21:51:41 07/19/25	
7	abc		This key note exist		21:51:49 07/19/25	
8	abc		Request sucessfull for : 995_789		21:52:13 07/19/25	
9	abc		Request sucessfull for : 995_789		21:54:00 07/19/25	
10	abc		Tentative on 995_789, not access		21:54:13 07/19/25	
11	abc		Conversation not exist		21:55:07 07/19/25	

Axes d'amélioration [0/2]

- Not Started Renforcement de la sécurité
- Not Started

